Week – 12 Project

MongoDB Basic Practice Task

Student ID: FSWD25095

Name: Dhanwanth JP

1. Database & Collection Setup:

- Create a new database (e.g., studentDB)
- Create a collection (e.g., students)

```
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\dhanw>mongosh
Current Mongosh Log ID: 69f773db5167b9a90aabc113
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB:          8.2.7
Using Mongosh:          2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/


To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

------
   The server generated these startup warnings when booting
   2026-05-03T20:48:21.049+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
------

test>

test>

test> use studentDB
switched to db studentDB
studentDB> db.createCollection("students")
{ ok: 1 }
studentDB> show dbs
admin      40.00 KiB
config     60.00 KiB
local      40.00 KiB
studentDB   8.00 KiB
studentDB> show collections
students
studentDB>
```

2. Insert Operations:

- Insert one document into the students collection.
- Insert multiple documents with fields like name, age, course, status.

```
studentDB> db.students.insertOne({
|   name: "Dhanwanth",
|   age: 22,
|   course: "MERN Stack",
|   status: "ongoing"
| })
{
  acknowledged: true,
  insertedId: ObjectId('69f775945167b9a90aabc114')
}
studentDB> db.students.insertMany([
|   { name: "Arun", age: 21, course: "MERN Stack", status: "ongoing" },
|   { name: "Priya", age: 23, course: "Data Science", status: "completed" },
|   { name: "Kiran", age: 20, course: "MERN Stack", status: "ongoing" },
|   { name: "Meena", age: 22, course: "Cyber Security", status: "pending" }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69f775b25167b9a90aabc115'),
    '1': ObjectId('69f775b25167b9a90aabc116'),
    '2': ObjectId('69f775b25167b9a90aabc117'),
    '3': ObjectId('69f775b25167b9a90aabc118')
  }
}
studentDB>
```

3. Read Operations:

- Fetch all documents from the collection.

```
studentDB>

studentDB> db.students.find()
[
  {
    _id: ObjectId('69f775945167b9a90aabc114'),
    name: 'Dhanwanth',
    age: 22,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f775b25167b9a90aabc115'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f775b25167b9a90aabc116'),
    name: 'Priya',
    age: 23,
    course: 'Data Science',
    status: 'completed'
  },
  {
    _id: ObjectId('69f775b25167b9a90aabc117'),
    name: 'Kiran',
    age: 20,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f775b25167b9a90aabc118'),
    name: 'Meena',
    age: 22,
    course: 'Cyber Security',
    status: 'pending'
  }
]
studentDB>
```

- Use filters to fetch specific data (e.g., students enrolled in “MERN Stack”).

```
studentDB> db.students.find({ course: "MERN Stack" })
[
  {
    _id: ObjectId('69f775945167b9a90aabc114'),
    name: 'Dhanwanth',
    age: 22,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f775b25167b9a90aabc115'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f775b25167b9a90aabc117'),
    name: 'Kiran',
    age: 20,
    course: 'MERN Stack',
    status: 'ongoing'
  }
]
studentDB>
```

4. Update Operations:

- Update a single document (e.g., change the status of a student to completed).
- Update multiple documents at once.

```
studentDB> db.students.updateOne(
|   { name: "Arun" },
|   { $set: { status: "completed" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
studentDB> db.students.updateMany(
|   { course: "MERN Stack" },
|   { $set: { status: "in progress" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
studentDB>
```

5. Delete Operations:

- Delete one document based on a condition.
- Delete all documents in the collection (for practice purpose only).

```
studentDB> db.students.deleteOne({ name: "Meena" })
{ acknowledged: true, deletedCount: 1 }
studentDB> db.students.deleteMany({})
{ acknowledged: true, deletedCount: 4 }
studentDB>
```

6. Query Operators:

- Practice using $gt, $lt, $in, $and, $or, $exists in different queries.

$gt

```
studentDB> db.students.find({ age: { $gt: 21 } })
[
  {
    _id: ObjectId('69f7779f5167b9a90aabc11a'),
    name: 'Priya',
    age: 23,
    course: 'Data Science',
    status: 'completed'
  },
  {
    _id: ObjectId('69f7779f5167b9a90aabc11c'),
    name: 'Meena',
    age: 22,
    course: 'Cyber Security',
    status: 'pending'
  }
]
studentDB>
```

$lt

```
studentDB> db.students.find({ age: { $lt: 22 } })
[
  {
    _id: ObjectId('69f7779f5167b9a90aabc119'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f7779f5167b9a90aabc11b'),
    name: 'Kiran',
    age: 20,
    course: 'MERN Stack',
    status: 'ongoing'
  }
]
studentDB>
```

$in

```
studentDB> db.students.find({ course: { $in: ["MERN Stack", "Data Science"] } })
[
  {
    _id: ObjectId('69f7779f5167b9a90aabc119'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f7779f5167b9a90aabc11a'),
    name: 'Priya',
    age: 23,
    course: 'Data Science',
    status: 'completed'
  },
  {
    _id: ObjectId('69f7779f5167b9a90aabc11b'),
    name: 'Kiran',
    age: 20,
    course: 'MERN Stack',
    status: 'ongoing'
  }
]
studentDB>
```

$and

```
studentDB> db.students.find({
|   $and: [
|     { course: "MERN Stack" },
|     { age: { $gt: 20 } }
|   ]
| })
[
  {
    _id: ObjectId('69f7779f5167b9a90aabc119'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  }
]
studentDB>
```

$or

```
studentDB> db.students.find({
|   $or: [
|     { course: "Cyber Security" },
|     { age: { $lt: 21 } }
|   ]
| })
[
  {
    _id: ObjectId('69f7779f5167b9a90aabc11b'),
    name: 'Kiran',
    age: 20,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f7779f5167b9a90aabc11c'),
    name: 'Meena',
    age: 22,
    course: 'Cyber Security',
    status: 'pending'
  }
]
studentDB>
```

$exists

```
studentDB> db.students.find({ status: { $exists: true } })
[
  {
    _id: ObjectId('69f7779f5167b9a90aabc119'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f7779f5167b9a90aabc11a'),
    name: 'Priya',
    age: 23,
    course: 'Data Science',
    status: 'completed'
  },
  {
    _id: ObjectId('69f7779f5167b9a90aabc11b'),
    name: 'Kiran',
    age: 20,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f7779f5167b9a90aabc11c'),
    name: 'Meena',
    age: 22,
    course: 'Cyber Security',
    status: 'pending'
  }
]
studentDB>
```

7. Use Case:

- Create a small use case like a Library System or E-Commerce product listing and apply insert, update, and search operations.

Create collection, insert products and search products.

```
studentDB> use ecommerce
switched to db ecommerce
ecommerce> db.createCollection("products")
{ ok: 1 }
ecommerce> db.products.insertMany([
|   { name: "Laptop", price: 60000, category: "Electronics", stock: 10 },
|   { name: "Phone", price: 25000, category: "Electronics", stock: 20 },
|   { name: "Shoes", price: 2000, category: "Fashion", stock: 50 }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69f7795a5167b9a90aabc11d'),
    '1': ObjectId('69f7795a5167b9a90aabc11e'),
    '2': ObjectId('69f7795a5167b9a90aabc11f')
  }
}
ecommerce> db.products.find({ category: "Electronics" })
[
  {
    _id: ObjectId('69f7795a5167b9a90aabc11d'),
    name: 'Laptop',
    price: 60000,
    category: 'Electronics',
    stock: 10
  },
  {
    _id: ObjectId('69f7795a5167b9a90aabc11e'),
    name: 'Phone',
    price: 25000,
    category: 'Electronics',
    stock: 20
  }
]
ecommerce>
```

Update stock, find expensive products and delete out-of-stock products.

```
]
ecommerce> db.products.updateOne(
|   { name: "Laptop" },
|   { $set: { stock: 8 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
ecommerce> db.products.find({ price: { $gt: 20000 } })
[
  {
    _id: ObjectId('69f7795a5167b9a90aabc11d'),
    name: 'Laptop',
    price: 60000,
    category: 'Electronics',
    stock: 8
  },
  {
    _id: ObjectId('69f7795a5167b9a90aabc11e'),
    name: 'Phone',
    price: 25000,
    category: 'Electronics',
    stock: 20
  }
]
ecommerce> db.products.deleteMany({ stock: 0 })
{ acknowledged: true, deletedCount: 0 }
ecommerce>
```